

Codes for importing the necessary libraries:

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.polynomial import Polynomial
```

Question 01 [4 Marks]

Represent the following polynomial using the **Polynomial class of the Numpy library**.

Let, $g(x) = -2x^5 + x^3 - 7x^2 + 3x - 9$

- A. [1 Mark] Find $3 \times g(-1) + 10$.
- B. [1.5 Marks] Find out the value of $2g'''(x) - g'(x) / 3$ at $x = 1$.
- C. [1.5 Marks] Plot $g(x)$ with a **red line** and $g''(x)$ (the second derivative) with a **green line** in the same graph $-4 \leq x \leq 5$.

Question 02 [8 Marks]

A set of data points/nodes : $(x_0, y_0), (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$

Let, $\text{data_X} = [x_0, x_1, x_2, \dots, x_n]$ and $\text{data_Y} = [y_0, y_1, y_2, \dots, y_n]$.

We define the **modified Lagrange interpolation polynomial** as:

$$P_n(x) = \sum_{k=0}^n y_{n-k} * l_k(x)$$

where y_{n-k} is the **reversed** order of the y-values, and $l_k(x)$ is the **modified Lagrange basis function** defined **only over odd-indexed** x-values:

$$l_k(x) = \prod_{j=0, j \neq k, j=1,3,5,7,\dots}^n \frac{(x-x_j)}{(x_k-x_j)}$$

Expanding, this gives:

$$P_n(x) = f(x_n) * l_0(x) + f(x_{n-1}) * l_1(x) + f(x_{n-2}) * l_2(x) + \dots + f(x_0) * l_n(x)$$

NOTE: The y-values are used in decreasing nodal order, while the basis product is restricted to odd-indexed x-nodes only ($j \in \{1, 3, 5, 7, \dots\}$).

- A. [4 Marks] Implement **l_k(data_X, k, x)**, where data_X are the nodal x-values, k is the Lagrange base number, and x is the value to evaluate at.
- B. [4 Marks] Implement **get_poly_lagrange(data_x, data_y, x_arr)** that computes $P_n(x)$ for every value in x_arr, using **l_k**. Here, data_X and data_Y are the nodal x and y values, and x_arr is the array of x value(s) to evaluate the interpolated polynomial at.

Sample Input	Sample Output
<pre>data_X = np.array([1, 3, 6, 10]) data_Y = np.array([4, 9, 12, 20]) x_arr = np.array([0, 2, 5, 8])</pre>	<pre>[26.26190476, 16.03174603, 6.1031746, 2.67460317]</pre>

Question 03 [8 Marks]

Suppose there is a function,

$$f(x) = x^5 + 3x^3 + 2x^2 - 2x + 15$$

To analyze the behavior of the Absolute Error when approximating the derivative of the function at $x = 5$ using **Backward Difference**, you have to determine the minimum value of h that ensures the absolute error stays below a given tolerance threshold of 0.002463. Additionally, calculate how many reductions of h are required to meet this condition.

As you already know,

A. Backward Difference, $f'(x) = \frac{f(x) - f(x-h)}{h}$

B. Absolute error = |Actual derivative - Approximate derivative|

You should start with an initial step size of $h = 0.5$ and iteratively reduce h by one-third, if necessary, until the error condition is satisfied.

Hint: You have to use the Polynomial class from the numpy library to define the function, $f(x)$ and your output should clearly state the **minimum h [4 Marks]** and the **total number of reductions [4 Marks]** required to achieve the desired accuracy.

Another Hint: In each iteration, update h by dividing it by 3 (i.e., $h = h / 3$). For example, if $h = 0.5$ in the current iteration, then in the next iteration $h = 0.5 / 3 \approx 0.1667$.

YOUR CODE SHOULD GIVE OUTPUT

Minimum h: $9.408382115794605 \times 10^{-07}$
Total number of reductions: 12